

5. Admissions in the MScFE

In this project, you conducted an experiment to help WQU improve enrollment in the Applied Data Science Lab. But let's not forget about our [Master of Science in Financial Engineering](#)! For your assignment, you'll help the MScFE conduct a similar experiment. This will be a great opportunity to put your new EDA, ETL, and statistics skills into action.

Also, keep in mind that for many of these submissions, you'll be passing in dictionaries that will test different parts of your code.

Note: Replace `ip_of_mongo_device` with your actual MongoDB IP address.

```
In [9]: from pymongo import MongoClient
        from pymongo.collection import Collection
        from teaching_tools.ab_test.reset import Reset

        # Import your libraries here
        import math
        import random

        import numpy as np
        import pandas as pd
        import plotly.express as px
        import scipy.stats

        from country_converter import CountryConverter
        from statsmodels.stats.contingency_tables import Table2x2
        from statsmodels.stats.power import GofChisquarePower
        from teaching_tools.ab_test.experiment import Experiment
```

Connect

 **Instruction:** Locate the IP address of the machine running MongoDB and assign it to the variable `host`. Make sure to use a **string** (i.e., wrap the IP in quotes).

 **Note:** The IP address is **dynamic** — it may change every time you start the lab. Always check the current IP before proceeding.

 MongoDB

```
In [10]: host = "192.181.124.2"
```

Task 7.5.1

```
In [11]: # Create `client`
client = MongoClient(host=host, port=27017)

# Create `db`
db = client["wqu-abtest"]

# Assign `mscfe-applicants` collection to `mscfe_app`
mscfe_app = db["mscfe-applicants"]
```

Explore

Task 7.5.2

```
In [12]: # Aggregate applicants by nationality
result = mscfe_app.aggregate(
    [
        {"$group": {"_id": "$countryISO2", "count": {"$sum": 1}}}
    ]
)

# Load result into DataFrame
df_nationality = (
    pd.DataFrame(result)
    .rename(columns={"_id": "country_iso2"})
    .sort_values("country_iso2")
    .reset_index(drop=True)
)

print("df_nationality type:", type(df_nationality))
print("df_nationality shape", df_nationality.shape)
df_nationality.head()
```

```
df_nationality type: <class 'pandas.core.frame.DataFrame'>
df_nationality shape (100, 2)
```

```
Out[12]:
```

	country_iso2	count
0	AF	15
1	AL	2
2	AO	1
3	AR	2
4	AU	1

Task 7.5.3

```
In [14]: # Instantiate `CountryConverter`
cc = CountryConverter()

# Create `country_name` column
df_nationality["country_name"] = cc.convert(
    df_nationality["country_iso2"], to="name_short"
)

# Create `country_iso3` column
```

```
df_nationality["country_iso3"] = cc.convert(
    df_nationality["country_iso2"], to="ISO3"
)

print("df_nationality type:", type(df_nationality))
print("df_nationality shape", df_nationality.shape)
df_nationality.head()
```

df_nationality type: <class 'pandas.core.frame.DataFrame'>
df_nationality shape (100, 4)

Out[14]:

	country_iso2	count	country_name	country_iso3
0	AF	15	Afghanistan	AFG
1	AL	2	Albania	ALB
2	AO	1	Angola	AGO
3	AR	2	Argentina	ARG
4	AU	1	Australia	AUS

Task 7.5.4

In [16]:

```
def build_nat_choropleth():
    fig = px.choropleth(
        data_frame=df_nationality,
        locations="country_iso3",
        color="count",
        projection="natural earth",
        color_continuous_scale=px.colors.sequential.Oranges,
        title="MScFE Applicants: Nationalities",
    )
    return fig

# Don't delete the code below 📌
nat_fig = build_nat_choropleth()
nat_fig.show()
```

ETL

In this section, you'll build a MongoRepository class. There are several tasks that will evaluate your class definition. You'll write your code in the cell below, and then submit each of those tasks one-by-one later on.

```
In [18]: class MongoRepository:
    """Repository class for interacting with MongoDB database."""

    # Task 7.5.5: `__init__` method
    def __init__(
        self,
        client=MongoClient(host=host, port=27017),
        db="wqu-abtest",
        collection="mscfe-applicants",
    ):
        self.collection = client[db][collection]

    # Task 7.5.6: `find_by_date` method
    def find_by_date(self, date_string):
        # Convert `date_string` to datetime object
        start = pd.to_datetime(date_string, format="%Y-%m-%d")
        # Offset `start` by one day
        end = start + pd.DateOffset(days=1)
        # Create PyMongo query for no-quiz applicants b/t `start` and `end`
        query = {
```

```

        "createdAt": {"$gte": start, "$lt": end},
        "admissionsQuiz": "incomplete",
    }
    # Query collection, get result
    result = self.collection.find(query)
    # Convert `result` to list
    observations = list(result)
    return observations

# Task 7.5.7: `update_applicants` method
def update_applicants(self, documents):
    # Initialize counters
    n = 0
    n_modified = 0
    # Iterate through `documents`
    for doc in documents:
        result = self.collection.update_one(
            filter={"_id": doc["_id"]},
            update={"$set": doc},
        )
        n += result.matched_count
        n_modified += result.modified_count
    # Create results dict
    transaction_result = {"n": n, "nModified": n_modified}
    return transaction_result

# Task 7.5.7: `assign_to_groups` method
def assign_to_groups(self, date):
    # Get observations
    observations = self.find_by_date(date)
    # Shuffle `observations`
    random.seed(42)
    random.shuffle(observations)
    # Get index position of item at observations halfway point
    idx = len(observations) // 2
    # Assign first half of observations to control group
    for doc in observations[:idx]:
        doc["inExperiment"] = True
        doc["group"] = "no email (c)"
    # Assign second half of observations to treatment group
    for doc in observations[idx:]:
        doc["inExperiment"] = True
        doc["group"] = "email (t)"
    # Update collection
    result = self.update_applicants(observations)
    return result

# Task 7.5.13: `find_exp_observations` method
def find_exp_observations(self):
    result = self.collection.find({"inExperiment": True})
    return list(result)

```

Task 7.5.5

```

In [19]: repo = MongoRepository()
print("repo type:", type(repo))
repo

```

repo type: <class '__main__.MongoRepository'>

Out[19]: <__main__.MongoRepository at 0x763979f72d10>

Task 7.5.6

```
In [21]: submission = repo.find_by_date("2022-06-01")  
submission
```

```
Out[21]: [{ '_id': ObjectId('6525d787953844722c837f21'),
  'createdAt': datetime.datetime(2022, 6, 1, 22, 7, 18),
  'firstName': 'Gail',
  'lastName': 'Weaver',
  'email': 'gail.weaver31@hotmail.com',
  'birthday': datetime.datetime(1996, 1, 10, 0, 0),
  'gender': 'female',
  'highestDegreeEarned': "Master's degree",
  'countryISO2': 'SG',
  'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c837f4a'),
  'createdAt': datetime.datetime(2022, 6, 1, 5, 30, 35),
  'firstName': 'Clint',
  'lastName': 'Zimmerman',
  'email': 'clint.zimmerman78@hotmail.com',
  'birthday': datetime.datetime(1987, 3, 11, 0, 0),
  'gender': 'male',
  'highestDegreeEarned': "Master's degree",
  'countryISO2': 'GH',
  'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c837f73'),
  'createdAt': datetime.datetime(2022, 6, 1, 22, 20, 4),
  'firstName': 'Nicholas',
  'lastName': 'Leon',
  'email': 'nicholas.leon30@gmail.com',
  'birthday': datetime.datetime(1962, 8, 8, 0, 0),
  'gender': 'male',
  'highestDegreeEarned': "Bachelor's degree",
  'countryISO2': 'PK',
  'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c837f8e'),
  'createdAt': datetime.datetime(2022, 6, 1, 13, 33, 8),
  'firstName': 'Lisa',
  'lastName': 'Rivera',
  'email': 'lisa.rivera19@hotmail.com',
  'birthday': datetime.datetime(1999, 1, 10, 0, 0),
  'gender': 'female',
  'highestDegreeEarned': "Bachelor's degree",
  'countryISO2': 'NG',
  'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c837fbd'),
  'createdAt': datetime.datetime(2022, 6, 1, 5, 34, 53),
  'firstName': 'Brian',
  'lastName': 'Lacroix',
  'email': 'brian.lacroix81@yahoo.com',
  'birthday': datetime.datetime(2004, 1, 13, 0, 0),
  'gender': 'male',
  'highestDegreeEarned': "Bachelor's degree",
  'countryISO2': 'SS',
  'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c838006'),
  'createdAt': datetime.datetime(2022, 6, 1, 3, 49, 49),
  'firstName': 'Debra',
  'lastName': 'Kettner',
  'email': 'debra.kettner53@yahoo.com',
  'birthday': datetime.datetime(1984, 8, 9, 0, 0),
  'gender': 'female',
  'highestDegreeEarned': "Bachelor's degree",
  'countryISO2': 'CI',
  'admissionsQuiz': 'incomplete'},
```

```
{'_id': ObjectId('6525d787953844722c838038'),
'createdAt': datetime.datetime(2022, 6, 1, 2, 38, 48),
'firstName': 'Alejandro',
'lastName': 'Castleman',
'email': 'alejandro.castleman47@microsoft.com',
'birthday': datetime.datetime(1999, 3, 31, 0, 0),
'gender': 'male',
'highestDegreeEarned': 'High School or Baccalaureate',
'countryISO2': 'PK',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c83805d'),
'createdAt': datetime.datetime(2022, 6, 1, 23, 5, 13),
'firstName': 'Everette',
'lastName': 'Howdeshell',
'email': 'everette.howdeshell73@microsoft.com',
'birthday': datetime.datetime(1995, 11, 10, 0, 0),
'gender': 'male',
'highestDegreeEarned': 'High School or Baccalaureate',
'countryISO2': 'TZ',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c838073'),
'createdAt': datetime.datetime(2022, 6, 1, 8, 23, 26),
'firstName': 'Brandon',
'lastName': 'Duty',
'email': 'brandon.duty93@hotmail.com',
'birthday': datetime.datetime(1995, 12, 8, 0, 0),
'gender': 'male',
'highestDegreeEarned': "Master's degree",
'countryISO2': 'BD',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c8380ac'),
'createdAt': datetime.datetime(2022, 6, 1, 8, 48, 10),
'firstName': 'Dorothy',
'lastName': 'Belk',
'email': 'dorothy.belk85@microsoft.com',
'birthday': datetime.datetime(1989, 5, 29, 0, 0),
'gender': 'female',
'highestDegreeEarned': "Bachelor's degree",
'countryISO2': 'SL',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c8380fc'),
'createdAt': datetime.datetime(2022, 6, 1, 7, 21, 51),
'firstName': 'Margaret',
'lastName': 'Robinson',
'email': 'margaret.robinson74@hotmail.com',
'birthday': datetime.datetime(1995, 7, 29, 0, 0),
'gender': 'female',
'highestDegreeEarned': 'Some College (1-3 years)',
'countryISO2': 'PK',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c8381fa'),
'createdAt': datetime.datetime(2022, 6, 1, 20, 14, 15),
'firstName': 'Cheri',
'lastName': 'Martin',
'email': 'cheri.martin75@gmail.com',
'birthday': datetime.datetime(1978, 5, 28, 0, 0),
'gender': 'female',
'highestDegreeEarned': 'Some College (1-3 years)',
'countryISO2': 'CM',
'admissionsQuiz': 'incomplete'},
```

```
{'_id': ObjectId('6525d787953844722c838204'),
'createdAt': datetime.datetime(2022, 6, 1, 23, 38, 33),
'firstName': 'David',
'lastName': 'Daniels',
'email': 'david.daniels26@hotmail.com',
'birthday': datetime.datetime(1998, 4, 11, 0, 0),
'gender': 'male',
'highestDegreeEarned': "Bachelor's degree",
'countryISO2': 'NG',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c838230'),
'createdAt': datetime.datetime(2022, 6, 1, 4, 14, 52),
'firstName': 'Miguel',
'lastName': 'Adams',
'email': 'miguel.adams47@microsoft.com',
'birthday': datetime.datetime(1997, 4, 12, 0, 0),
'gender': 'male',
'highestDegreeEarned': "Bachelor's degree",
'countryISO2': 'ZA',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c838243'),
'createdAt': datetime.datetime(2022, 6, 1, 3, 56, 53),
'firstName': 'Jason',
'lastName': 'Davis',
'email': 'jason.davis91@yahoo.com',
'birthday': datetime.datetime(1990, 10, 7, 0, 0),
'gender': 'male',
'highestDegreeEarned': "Master's degree",
'countryISO2': 'NG',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c838284'),
'createdAt': datetime.datetime(2022, 6, 1, 5, 36, 55),
'firstName': 'Judson',
'lastName': 'Thacker',
'email': 'judson.thacker63@yahoo.com',
'birthday': datetime.datetime(1989, 2, 3, 0, 0),
'gender': 'male',
'highestDegreeEarned': 'High School or Baccalaureate',
'countryISO2': 'NG',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c838299'),
'createdAt': datetime.datetime(2022, 6, 1, 3, 35, 1),
'firstName': 'Susan',
'lastName': 'Franklin',
'email': 'susan.franklin58@hotmail.com',
'birthday': datetime.datetime(1997, 1, 29, 0, 0),
'gender': 'female',
'highestDegreeEarned': 'Some College (1-3 years)',
'countryISO2': 'IN',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c8382ff'),
'createdAt': datetime.datetime(2022, 6, 1, 7, 44, 56),
'firstName': 'Reynaldo',
'lastName': 'Sutherland',
'email': 'reynaldo.sutherland76@yahoo.com',
'birthday': datetime.datetime(2004, 6, 27, 0, 0),
'gender': 'male',
'highestDegreeEarned': "Bachelor's degree",
'countryISO2': 'SO',
'admissionsQuiz': 'incomplete'},
```

```
{'_id': ObjectId('6525d787953844722c838314'),
'createdAt': datetime.datetime(2022, 6, 1, 12, 3, 24),
'firstName': 'Malcolm',
'lastName': 'Marshall',
'email': 'malcolm.marshall41@gmail.com',
'birthday': datetime.datetime(1995, 8, 22, 0, 0),
'gender': 'male',
'highestDegreeEarned': "Master's degree",
'countryISO2': 'ER',
'admissionsQuiz': 'incomplete'},
{'_id': ObjectId('6525d787953844722c8383bc'),
'createdAt': datetime.datetime(2022, 6, 1, 6, 2, 5),
'firstName': 'Gail',
'lastName': 'Harman',
'email': 'gail.harman37@gmail.com',
'birthday': datetime.datetime(1993, 9, 4, 0, 0),
'gender': 'female',
'highestDegreeEarned': "Bachelor's degree",
'countryISO2': 'PK',
'admissionsQuiz': 'incomplete'}}
```

Task 7.5.7

```
In [23]: date = "2022-06-02"
submission = repo.assign_to_groups(date)
submission
```

```
Out[23]: {'n': 9, 'nModified': 9}
```

Experiment

Prepare Experiment

Task 7.5.8

```
In [25]: # Instantiate power calculator
chi_square_power = GofChisquarePower()

# Calculate group size
group_size = math.ceil(
    chi_square_power.solve_power(effect_size=0.5, alpha=0.05, power=0.8)
)

print("Group size:", group_size)
print("Total # of applicants needed:", group_size * 2)
```

Group size: 32

Total # of applicants needed: 64

Task 7.5.9

```
In [27]: # Aggregate no-quiz applicants by sign-up date
result = mscfe_app.aggregate(
    [
        {"$match": {"admissionsQuiz": "incomplete"}},
        {
            "$group": {
                "_id": {"$dateTrunc": {"date": "$createdAt", "unit": "day"}},
```

```

        "new_users": {"$count": {}},
    },
]
)

# Load result into DataFrame / Series
no_quiz_mscfe = (
    pd.DataFrame(result)
    .rename(columns={"_id": "date"})
    .set_index("date")
    .sort_index()
    .squeeze()
)
no_quiz_mscfe.index.name = "date"

print("no_quiz type:", type(no_quiz_mscfe))
print("no_quiz shape:", no_quiz_mscfe.shape)
no_quiz_mscfe.head()

```

```

no_quiz type: <class 'pandas.core.series.Series'>
no_quiz shape: (30,)

```

```

Out[27]: date
2022-06-01    20
2022-06-02     9
2022-06-03    12
2022-06-04    15
2022-06-05    11
Name: new_users, dtype: int64

```

Task 7.5.10

```

In [29]: mean = no_quiz_mscfe.describe()["mean"]
std = no_quiz_mscfe.describe()["std"]
print("no_quiz mean:", mean)
print("no_quiz std:", std)

```

```

no_quiz mean: 12.133333333333333
no_quiz std: 3.170264139254595

```

Ungraded Task: Complete the code below so that it calculates the mean and standard deviation of the probability distribution for the total number of days assigned to `exp_days`.

```

In [31]: exp_days = 7
sum_mean = mean * exp_days
sum_std = std * np.sqrt(exp_days)
print("Mean of sum:", sum_mean)
print("Std of sum:", sum_std)

```

```

Mean of sum: 84.93333333333334
Std of sum: 8.3877305028539

```

Task 7.5.11

```

In [32]: prob_65_or_fewer = scipy.stats.norm.cdf(
    group_size * 2, loc=sum_mean, scale=sum_std
)
prob_65_or_greater = 1 - prob_65_or_fewer

```

```
print(  
    f"Probability of getting 65+ no_quiz in {exp_days} days:",  
    round(prob_65_or_greater, 3),  
)
```

Probability of getting 65+ no_quiz in 7 days: 0.994

Run Experiment

Task 7.5.12

```
In [34]: exp = Experiment(repo=client, db="wqu-abtest", collection="mscfe-applicants")  
exp.reset_experiment()  
result = exp.run_experiment(days=exp_days, assignment=True)  
print("result type:", type(result))  
result
```

result type: <class 'dict'>

```
Out[34]: {'acknowledged': True, 'inserted_count': 307}
```

Analyze Results

Task 7.5.13

```
In [36]: submission = repo.find_exp_observations()  
submission[:3]
```

```

Out[36]: [{ '_id': ObjectId('6a65dc1efcd458b34ae498ae'),
  'createdAt': datetime.datetime(2026, 7, 29, 17, 0, 58),
  'firstName': 'Nathan',
  'lastName': 'Palmer',
  'email': 'nathan.palmer79@hotmail.com',
  'birthday': datetime.datetime(1996, 11, 7, 0, 0),
  'gender': 'male',
  'highestDegreeEarned': "Bachelor's degree",
  'countryISO2': 'ZW',
  'admissionsQuiz': 'complete',
  'inExperiment': True,
  'group': 'email (t)'},
{'_id': ObjectId('6a65dc1efcd458b34ae498b6'),
 'createdAt': datetime.datetime(2026, 8, 1, 22, 11, 16),
 'firstName': 'Richard',
 'lastName': 'Neubert',
 'email': 'richard.neubert58@yahoo.com',
 'birthday': datetime.datetime(1980, 1, 11, 0, 0),
 'gender': 'male',
 'highestDegreeEarned': "Master's degree",
 'countryISO2': 'NG',
 'admissionsQuiz': 'complete',
 'inExperiment': True,
 'group': 'email (t)'},
{'_id': ObjectId('6a65dc1efcd458b34ae498b7'),
 'createdAt': datetime.datetime(2026, 7, 28, 16, 2, 5),
 'firstName': 'Philip',
 'lastName': 'Granthan',
 'email': 'philip.granthan70@hotmail.com',
 'birthday': datetime.datetime(1999, 3, 9, 0, 0),
 'gender': 'male',
 'highestDegreeEarned': "Master's degree",
 'countryISO2': 'TN',
 'admissionsQuiz': 'complete',
 'inExperiment': True,
 'group': 'email (t)'}]

```

Task 7.5.14

```

In [38]: result = repo.find_exp_observations()
df = pd.DataFrame(result)
print("df type:", type(df))
print("df shape:", df.shape)
df.head()

```

```

df type: <class 'pandas.core.frame.DataFrame'>
df shape: (66, 12)

```

Out[38]:

	_id	createdAt	firstName	lastName	
0	6a65dc1efcd458b34ae498ae	2026-07-29 17:00:58	Nathan	Palmer	nathan.palmer79@hotmail.com
1	6a65dc1efcd458b34ae498b6	2026-08-01 22:11:16	Richard	Neubert	richard.neubert58@yahoo.com
2	6a65dc1efcd458b34ae498b7	2026-07-28 16:02:05	Philip	Grantham	philip.grantham70@hotmail.com
3	6a65dc1efcd458b34ae498bc	2026-08-01 18:55:03	James	Steele	james.steele9@yahoo.com
4	6a65dc1efcd458b34ae498be	2026-07-27 15:23:11	Travis	Desroches	travis.desroches88@gmail.com

Task 7.5.15

```
In [40]: data = pd.crosstab(
        index=df["group"],
        columns=df["admissionsQuiz"],
        normalize=False,
    )
    print("data type:", type(data))
    print("data shape:", data.shape)
    data
```

data type: <class 'pandas.core.frame.DataFrame'>
data shape: (2, 2)

Out[40]: **admissionsQuiz** **complete** **incomplete**

group		
email (t)	7	26
no email (c)	1	32

Task 7.5.16

```
In [42]: def build_contingency_bar():
        fig = px.bar(
            data_frame=data,
            barmode="group",
            title="MScFE: Admissions Quiz Completion by Group",
        )
        fig.update_layout(
            xaxis_title="Group",
            yaxis_title="Frequency [count]",
            legend={"title": "Admissions Quiz"},
        )
        return fig
```

```
# Don't delete the code below 📌  
cb_fig = build_contingency_bar()  
cb_fig.show()
```

Task 7.5.17

```
In [44]: contingency_table = Table2x2(data.values)  
print("contingency_table type:", type(contingency_table))  
contingency_table.table_orig
```

contingency_table type: <class 'statsmodels.stats.contingency_tables.Table2x2'>

```
Out[44]: array([[ 7, 26],  
               [ 1, 32]])
```

Task 7.5.18

```
In [46]: chi_square_test = contingency_table.test_nominal_association()  
print("chi_square_test type:", type(chi_square_test))  
print(chi_square_test)
```

chi_square_test type: <class 'statsmodels.stats.contingency_tables._Bunch'>
df 1
pvalue 0.023642218909198598
statistic 5.120689655172414

Task 7.5.19

```
In [48]: odds_ratio = contingency_table.oddsratio.round(1)
print("Odds ratio:", odds_ratio)
```

Odds ratio: 8.6

Copyright 2024 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.